

UAM OIDC Android App Development Using AppAuth Guide - 6.0.1-GA

Table of Contents

1. Preface - UAM OIDC Android App Development Using AppAuth Guide.....	3
2. Introduction to UAM OIDC Android App Development Using AppAuth.....	4
3. OAuth Client Registration.....	6
4. OIDC Android App Creation.....	7
5. Test/Build OIDC Android APK.....	29
6. Appendices.....	30

1. Preface - UAM OIDC Android App Development Using AppAuth Guide

This document guides you on the building of an OIDC Android App using the AppAuth client SDK to authenticate with the AccessMatrix UAM server.

Intended Reader

This guide is intended as a reference for experienced application developers.

Document Scope

This document only provides the necessary steps to create an OIDC App using AppAuth for Android and will not provide documentation on the configuration of the AccessMatrix UAM server as the OpenID Provider.

For feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- [AccessMatrix Common Deployment Guide](#)
- [AccessMatrix Common Login Page Deployment Guide](#)
- [UAM OAuth and OIDC Implementation Guide](#)

2. Introduction to UAM OIDC Android App Development Using AppAuth

One approach to have Mobile SSO or externalized authentication is to use OpenID Connect.

AccessMatrix UAM Server and Common Login Page web application act as the OpenID Connect provider whose role is to authenticate the user and to issue access token to the mobile app. Mobile app has the role of Relying Party that uses the access token to get the application session or to get application resources from its backend services (resource server).

It is recommended that mobile app, a native app, uses the Proof Key for Code Exchange (PKCE) Authorization Code Flow to authenticate and authorize its user before they can gain access to their data. This flow normally uses the mobile browser (or custom tab) to access the OpenID Connect Provider in order to authenticate the user. With multiple mobile apps, authentication should only be done once if the mobile apps are accessed while the OpenID Connect provider's user session is still valid.

AppAuth for Android is a client SDK, loaded by the mobile app, for communicating with OAuth 2.0 and OpenID Connect providers. It maps the requests and responses according to the specifications and also provides convenience methods to assist with common tasks like performing an action to refresh tokens. This library follows the best practices set out in OAuth 2.0 for Native Apps including using Custom Tabs for the auth request, while conforming to PKCE extension to OAuth to secure public clients when custom URI scheme redirects are used.

AccessMatrix UAM Server is a certified OpenID Provider and it supports Native Apps through custom URI scheme redirects.

This guide will walk you through the following steps to create an OIDC Android App to authenticate with the AccessMatrix UAM server (the OpenID Provider):

- [Register OpenID Client on the AccessMatrix UAM Server](#)
- [Create OIDC Android App based on AppAuth](#)
- [Test/build the Android APK](#)



Note: The purpose of this guide is to walk through building the OIDC Android App. Thus, some parts, such as storing configuration and authenticate state, are simplified. For an actual production app, you may want to adapt it based on your app configuration and authentication state could also be encrypted (by Android Keystore) when stored in the app database/file.

Prerequisites

- An AccessMatrix UAM Server that is up and ready to serve OIDC Authentication requests.

Below are the installation references of AccessMatrix UAM Server as the OpenID Provider:

- Read the [AccessMatrix Common Deployment Guide](#) for installation of AccessMatrix UAM Server.
- Read the [AccessMatrix Common Login Page Deployment Guide](#) to prepare AccessMatrix UAM Server as the OpenID Provider:
 - Set up OpenID Provider Front-end Server (refer to [AccessMatrix Common Login Page Deployment Guide - AccessMatrix CLP Deployment](#))
 - Set up OpenID Provider and OAuth Client for the Android App (refer to [AccessMatrix Common Login Page Deployment Guide - CLP as OIDC OpenID Provider](#))



Note: Ensure that the https URI is set for the **OpenID Provider Issuer** attribute of your **OAuth OAuthz Server** configurations in the Admin Console.

- Configure authentication method for Android Mobile App (refer to [AccessMatrix Common Login Page Deployment Guide - CLP Authentication Methods](#))
- Android Studio with virtual/ real device (downloadable [here](#))

3. OAuth Client Registration

Register OAuth Client in Admin Console

Refer to [UAM OAuth and OIDC Implementation Guide - Configure Authorization Code Flow with PKCE for Native Application](#) for the steps to register the OIDC Android App as OAuth Client.

The following are the fields that you will see during the OAuth Client registration:

- **OAuth client_id:** a system-generated value that uniquely identify your Android app to the OpenID Provider. You also need to set this value as part of Android App [Configuration](#).
- **OAuth redirect URIs:** redirectUri is a URI residing in the OIDC Android App and this value needs to be configured there. For example, "com.isprint.appauth.demo://oauth2redirect". This URI is used by the browser to perform redirection to the mobile app after authorization request gets performed successfully to the OpenID Provider. It is a mandatory field when setting up OAuth Client in AccessMatrix's Admin Console and configuring Android App [Configuration](#). Click [here](#) to learn how and where to configure this value.

4. OIDC Android App Creation

Android Studio is the official IDE for Android App development. You may download and find out more about the product features, documentations, and other information [here](#).

Android SDK Specification

This project requires Android 9 and API level 28 (Pie) to compile and the produced binaries require a minimum of Android 7 and API level 24 (Nougat) to be used.



Note: Make sure you have downloaded the corresponding SDKs using Android Studio's SDK Manager before proceeding to the next sections.

Create New Project

1. Open Android Studio and click **File > New > New Project**. It will prompt to **Select a Project Template**. Click **Next** to proceed, then select "Java" as the **Language**, and choose "API 24:Android 7.0 (Nougat)" as the **Minimum SDK**.
2. (Optional) Configure this to let Android Studio import all used (java) classes automatically for you:
 - a. Click **File > Settings > Editor > General > Auto Import** in the popup box.
 - b. On the right panel, under the **Java** header, choose "All" from the **Insert imports on paste** drop-down list. Check **Add unambiguous imports on the fly** and all other check boxes.
3. Configure the compiled SDK version by clicking **File > Project Structure > Modules**. On the right panel, under **Properties**, set "28" as the **Compile Sdk Version**. Set JDK 1.8 for **Source Compatibility and Target Compatibility**.
4. Click **Default Config**, you should see "24" as the **Min SDK Version**. For the **Target SDK Version**, you may choose "24" too.

Import Libraries

Navigate to the **build.gradle** file of your project and add the following library dependencies for this app:

```
Code
```

```
dependencies {
    implementation 'net.openid:appauth:0.8.1'
    implementation 'joda-time:joda-time:2.0'
}
```

net.openid:appauth:0.8.1 is the library for AppAuth library, while joda-time:joda-time:2.0 is used for formatting the date and time of OIDC tokens for UI display.

Configure OIDC Client

Now the project is ready to use AppAuth library for development. Prior to that, configure your app as an OIDC client with the mandatory and optional fields as shown in the code snippets below:

Java

```
....
public final class Configuration {

    //TODO: mandatory settings
    private static final String discoveryUri =
"https://4a04e2712f87.ngrok.io/clp/.well-known/openid-configuration";
    //"http://192.168.1.44:8080/clp/.well-known/openid-configuration";
    private static final String redirectUri =
"com.isprint.appauth.demo://oauth2redirect";
    static final String mClientId =
"QEVx6ON3n19E0j618aXfuQ"; //"rkMWSrOQiHmcvnbLtPhUvQ";

    //TODO: optional setting
    static final String mScope = "openid";

    static final ConnectionBuilder mConnectionBuilder =
DefaultConnectionBuilder.INSTANCE;
    //for testing purpose use this if your OP config
and endpoints are using HTTP:
    //static final ConnectionBuilder mConnectionBuilder
= ConnectionBuilderForTesting.INSTANCE;

    ...
}
```

Click [here](#) for the full source.

Fields	Description	Mandatory (Y/N)
discoveryUri	The AccessMatrix UAM server is the OpenID Provider (OP) hence, the OP configuration would be obtained from the AccessMatrix UAM server's CLP discovery URI. By default, the uri would look like this: https://IDPHostName/clp/.well-known/openid-configuration where clp is the default context path for the CLP deployment, the front-end server of OpenID Server.	
redirectUri	redirectUri is the URI where browser will be redirected to after authorization request is processed successfully to OP. As recommended, the Custom tab is used to trigger the authorization request and redirect the browser back to the app according to the given redirectUri. Click here for configuration details.	Y
mClientId	This is the OAuth client_id which can be obtained from client registration .	Y
mScope	Default value is openid. More scopes can be added accordingly and it should be separated with a space as the delimiter, e.g. openid profile.	N
ConnectionBuilder	ConnectionBuilder is used to create instances of HTTP Connection to directly interact with OP (using authorization service) endpoints, i.e. those not performed via a browser - token exchange, token refresh, etc. Click here for more details.	N

redirectUri

How to configure?

- **Redirect URI scheme**

It is recommended to use the custom scheme based redirect URI (i.e. those of "my.scheme:/path" forms), as this is the most widely supported across all versions of Android.

- **Naming Convention**

It is strongly recommended to use "reverse domain name notation", which is a naming

convention based on the domain name system, but where the domain components are reversed. For example, if you declare a custom scheme "com.isprint.appauth.demo" and host "/oauth2redirect", then the redirect URL will look like: *com.isprint.appauth.demo://oauth2redirect*. Click [here](#) to find out more about custom scheme.

Where to configure?

Assuming "com.isprint.appauth.demo://oauth2redirect" is the custom scheme to be defined, navigate to your app's manifest file (**AndroidManifest.xml**) and **build.gradle** file to perform the following changes:

- Under the **build.gradle** file, add your custom scheme in your app as shown in the XML snippets below:

Code

```
android {
    ...

    defaultConfig {
        ...
        manifestPlaceholders = [
            'appAuthRedirectScheme':
'com.isprint.appauth.demo'
        ]
    }
    ...
}
```

- In the app's manifest file (**AndroidManifest.xml**), declare the custom scheme based redirect URI with the android:scheme and android:host values, as shown in the snippets of the file below:

XML

```
<activity
android:name="net.openid.appauth.RedirectUriReceiverActivity">
    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>
        <category
android:name="android.intent.category.DEFAULT"/>
        <category
android:name="android.intent.category.BROWSABLE"/>
        <data android:scheme="com.isprint.appauth.demo"
            android:host="/oauth2redirect"/>
    </intent-filter>
</activity>
```

ConnectionFactory

ConnectionFactory is an AppAuth interface to create connection for the purpose of calling OP endpoints.

- Leave the configuration as it is if your OP configuration and endpoints are using HTTPS protocol:

Java

```
static final ConnectionBuilder mConnectionFactory =
DefaultConnectionFactory.INSTANCE;
//static final ConnectionBuilder mConnectionFactory
= ConnectionBuilderForTesting.INSTANCE;
```

- Due to security reasons, HTTP protocol should not be used in production environment. However, if HTTP protocol is used for testing purpose, uncomment the existing configuration as shown below:

Java

```
//static final ConnectionBuilder mConnectionFactory
= DefaultConnectionFactory.INSTANCE;
static final ConnectionBuilder mConnectionFactory =
ConnectionFactoryForTesting.INSTANCE;
```

- ConnectionBuilder can be further customized to meet your requirement, e.g. for certificate pinning, adjusting timeout or adding some additional certificates to the trusted set (such as for testing).
 - This can be achieved in AppAuth by providing a custom ConnectionBuilder instance. Click [here](#) to find out more.

Implement PKCE Authorization Code Flow

This flow consists of four stages:

1. Discovering endpoints to interact with OpenID Provider and AccessMatrix UAM server.
 2. Authorizing mobile user via browser to obtain authorization code.
 3. Exchange of authorization code to obtain OIDC tokens (access Token, Id Token and optionally, refresh token too).
 4. Using access token to interact with resource server to further access user data.
-

AppAuth library supports native application interaction with OP and it keeps track of the authorization state of the session (e.g. the authorization, token request and response) in the `AuthState` class. Typically, `AuthState` should be stored encrypted in `SharedPreferences` or in some other persistent store private to the app. For the purpose of illustration, the `AppState` will be stored in the `SharePreferences`, private to the app using the `AuthStateManager` class. Refer to [AuthStateManager](#) for the full source.

This section will guide you through using AppAuth to create an OIDC Android App that performs OIDC authentication with mobile user via browser, followed by token exchange to obtain OIDC tokens, then accessing the resource server.

- [Initialize AppAuth Services](#)
- [Prepare Auth Request](#)
- [Trigger Auth Request](#)
- [Get OIDC Tokens After Token Exchange](#)
- [Access Resource Server](#)

In addition, the following topics will also be covered here:

- [Manage User State for Mobile SSO](#)
- [Refresh Token](#)

Initialize AppAuth Services

Prerequisite:

Make sure that the corresponding classes have been added to your android project:

- A login activity class, that will embark the OIDC authentication for your mobile app (e.g. `LoginActivity.java`)
- An OIDC Client configuration of your app (Refer to [Configuration](#) section for the details.)
- [AuthStateManager](#)
- [ConnectionBuilderForTesting](#) (if required for testing use)

Next, perform the following:

1. Initialize instances of `ExecutorService`, `AuthStateManager` and `Configuration`. `ExecutorService` is used to run task(s) in asynchronous mode.

Java

```
package com.isprint.appauth.demo;
...
import java.util.concurrent.ExecutorService;
...
public class LoginActivity extends AppCompatActivity {
    ...
    private Configuration mConfig;
    private AuthStateManager mAuthStateManager;
    private ExecutorService mExecutor;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        ...
        mExecutor =
        Executors.newSingleThreadExecutor();
        mConfig = Configuration.getInstance(this);
        mAuthStateManager =
        AuthStateManager.getInstance(this);
        ...
    }
    ...
}
```

2. Run the `initializeEnvironmentPrepareAuthRequest` method in asynchronous mode to trigger AppAuth related services and configuration.

Java

```
...
public class LoginActivity extends AppCompatActivity {
    ...
    private ExecutorService mExecutor;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        ...
        Log.i(TAG, "Initializing environment for
authentication");
        mExecutor.submit(this::initializeEnvironmentPrepareAuthRequest);
    }
    ...
}
```

- a. `AuthorizationService` is an AppAuth's service that triggers browser-based mobile user authorization. Together with Auth request (`mAuthRequest`) and Custom Tab Intent (`mAuthIntent`), this service needs to be recreated before each use. Chrome and Samsung custom tabs will be used as the default when creating `AuthorizationService`.

The following is optional and only used for testing:

-
- `AppAuthConfiguration.Builder` can be configured to skip the validation checking that requires the OP's issuer to be HTTPS, e.g.
`builder.setSkipIssuerHttpsCheck(true);`. For production, this should be set to `"false"`.
 - `ConnectionBuilderForTesting` can be configured as the value for `Configuration.mConnectionBuilder` when OP's endpoints are configured to be using non-HTTPS protocol. For production, this should be set to `DefaultConnectionBuilder`.

Java

```
...
public class LoginActivity extends AppCompatActivity {
    ...
    private AuthorizationService mAuthService;
    private final AtomicReference<AuthorizationRequest>
mAuthRequest = new AtomicReference<>();
    private final AtomicReference<CustomTabsIntent>
mAuthIntent = new AtomicReference<>();

    private void
initializeEnvironmentPrepareAuthRequest() {
        recreateAuthorizationService();
        ...
    }

    private void recreateAuthorizationService() {
        if (mAuthService != null) {
            Log.i(TAG, "Discarding existing AuthService
instance");
            mAuthService.dispose();
        }
        mAuthService = createAuthorizationService();
        mAuthRequest.set(null);
        mAuthIntent.set(null);
    }

    private AuthorizationService
createAuthorizationService() {
        Log.i(TAG, "Creating authorization service");
        AppAuthConfiguration.Builder builder = new
AppAuthConfiguration.Builder();
        builder.setBrowserMatcher(new BrowserAllowList(
VersionedBrowserMatcher.CHROME_CUSTOM_TAB,
VersionedBrowserMatcher.SAMSUNG_CUSTOM_TAB));
        builder.setConnectionBuilder(mConfig.mConnectionBuilder);
    }
}
```

```

        //In Production: need to set this to false
        builder.setSkipIssuerHttpsCheck(true);
        builder.setSkipNonceVerification(false);

        return new AuthorizationService(this,
builder.build());
    }
    ...
}

```

- b. Now that `AuthorizationService` is created, it requires OpenID Config from OP's discovery endpoint. If this is the first time the app is loaded, `AuthorizationServiceConfiguration` will fetch OpenID Config and have the results stored in `AuthStateManager`. This is to make `AuthStateManager` ready to be reused in the event of applications switching or when the app is closed and reopened.

Java

```

...
public class LoginActivity extends AppCompatActivity {
    ...
    private void initializeEnvironmentPrepareAuthRequest() {
        ...

        if
(mAuthStateManager.getCurrent().getAuthorizationServiceConfigu
ration() != null) {
            Log.i(TAG, "App was running not long ago, auth
config already established");
            initializeClientAndAuthRequest();
            return;
        }

        Log.i(TAG, "Retrieving OpenID discovery doc");
        AuthorizationServiceConfiguration.fetchFromUrl(
            mConfig.mDiscoveryUri,
            this::handleConfigurationRetrievalResult,
            mConfig.mConnectionBuilder);
    }

    ...

    @MainThread
    private void handleConfigurationRetrievalResult(
        AuthorizationServiceConfiguration config,
        AuthorizationException ex) {
        if (config == null) {
            Log.i(TAG, "Failed to retrieve discovery
document", ex);
            return;
        }
    }
}

```

```

        Log.i(TAG, "Discovery document retrieved");
        mAuthStateManager.replace(new AuthState(config));

mExecutor.submit(this::initializeClientAndAuthRequest);
    }
}

```

Prepare Auth Request

1. Once AppAuth Services and Config are ready, the app will proceed to create an authorization request for OIDC client via `initializeClientAndAuthRequest` method.

Java

```

...
public class LoginActivity extends AppCompatActivity {
    ...
    private final AtomicReference<AuthorizationRequest>
mAuthRequest = new AtomicReference<>();

    private void initializeEnvironmentPrepareAuthRequest() {

        ...
        if
(mAuthStateManager.getCurrent().getAuthorizationServiceConfiguratio
n() != null) {
            Log.i(TAG, "App was running not long ago, auth config
already established");
            initializeClientAndAuthRequest();
            return;
        }

        Log.i(TAG, "Retrieving OpenID discovery doc");
        AuthorizationServiceConfiguration.fetchFromUrl(
            mConfig.mDiscoveryUri,
            this::handleConfigurationRetrievalResult,
            mConfig.mConnectionBuilder);
    }

    @MainThread
    private void handleConfigurationRetrievalResult(
        AuthorizationServiceConfiguration config,
        AuthorizationException ex) {
        if (config == null) {
            Log.i(TAG, "Failed to retrieve discovery document",
ex);
            return;
        }

        Log.i(TAG, "Discovery document retrieved");
        mAuthStateManager.replace(new AuthState(config));
        mExecutor.submit(this::initializeClientAndAuthRequest);
    }
}

```

```

    }

    ...

    private void initializeClientAndAuthRequest() {
        Log.i(TAG, "client ID: " + mConfig.mClientId);
        runOnUiThread(this::initializeAuthRequest);
        return;
    }

    private void initializeAuthRequest() {
        createAuthRequest();
        warmUpBrowser();
    }

    private void createAuthRequest() {
        Log.i(TAG, "Preparing auth request");
        AuthorizationRequest.Builder authRequestBuilder = new
        AuthorizationRequest.Builder(
            mAuthStateManager.getCurrent().getAuthorizationServiceConfiguration
            (),
                mConfig.mClientId,
                ResponseTypeValues.CODE,
                mConfig.mRedirectUri)
                .setScope(mConfig.mScope);

        mAuthRequest.set(authRequestBuilder.build());
    }
    ...
}

```

2. This process involves preparing auth request, where most of the request parameter values are populated from the settings in the Configuration object. This is followed by warming up custom tabs intent so that the browser is ready when OIDC authorization is triggered by the mobile user. `CountDownLatch` is used here to ensure custom tab is ready when a user authentication event is triggered.

Java

```

...
import androidx.browser.customtabs.CustomTabsIntent;

public class LoginActivity extends AppCompatActivity {

    ...
    private final AtomicReference<CustomTabsIntent> mAuthIntent =
    new AtomicReference<>();

    private CountDownLatch mAuthIntentLatch = new
    CountDownLatch(1);

```

```

...
private void initializeAuthRequest() {
    createAuthRequest();
    warmUpBrowser();
}

...

private void warmUpBrowser() {
    mAuthIntentLatch = new CountDownLatch(1);
    mExecutor.execute(() -> {
        Log.i(TAG, "Warming up browser instance for auth
request");
        CustomTabsIntent.Builder intentBuilder =
mAuthService.createCustomTabsIntentBuilder(mAuthRequest.get().toUri
());
        mAuthIntent.set(intentBuilder.build());
        mAuthIntentLatch.countDown();
    });
}
}

```

Trigger Auth Request

In this sample code, auth request is triggered by a login button and that makes `mAuthService.performAuthorizationRequest` send an auth request to the OP via custom tab/browser. There are two `PendingIntents` to be specified in this method:

- `completionIntent` is the intent that would continue to receive the response of the authorization request (to proceed with the token exchange in the event that the authorization request is performed successfully).
- `cancelIntent` is the intent that would be redirected to when mobile user cancels the authorization request (by clicking on the **Cancel** button on the browser/custom tab).

Java

```

...

public class LoginActivity extends AppCompatActivity {

    ...

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        ...
        Button loginBtn = (Button)

```

```
findViewById(R.id.loginBtn);
loginBtn.setClickable(true);
loginBtn.setOnClickListener(new
View.OnClickListener() {
    @Override
    public void onClick(View view) {
        startAuth();
    }
});

...
}

private void startAuth () {
    mExecutor.submit(this::doAuth);
}

private void doAuth() {

    try {
        //this is to make sure custom tab is ready
        before proceed with auth request
        mAuthIntentLatch.await();
    } catch (InterruptedException ex) {
        Log.w(TAG, "Interrupted while waiting for
        auth intent");
    }

    //the intent to be sent when the flow
    completes.
    Intent completionIntent = new Intent(this,
    TokenActivity.class);
    //the intent to be sent when the flow cancels.
    Intent cancelIntent = new Intent(this,
    LoginActivity.class);
    cancelIntent.putExtra(EXTRA_FAILED, true);

    cancelIntent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);

    mAuthService.performAuthorizationRequest(
        mAuthRequest.get(),
        PendingIntent.getActivity(this, 0,
        completionIntent, 0),
        PendingIntent.getActivity(this, 0,
        cancelIntent, 0),
        mAuthIntent.get());
}
...
}
```

Refer to [LoginActivity](#) for the full source.

Get OIDC Tokens After Token Exchange

For the purpose of illustration, the TokenActivity class is used as the intent that received the response of the authorization request.

Initialize AppAuth Services

Like LoginActivity class, TokenActivity class is also expected to initialize AppAuth services and config as shown in the code snippets below:

Java

```
package com.isprint.appauth.demo;

...

import net.openid.appauth.AppAuthConfiguration;
import net.openid.appauth.AuthState;
import net.openid.appauth.AuthorizationException;
import net.openid.appauth.AuthorizationResponse;
import net.openid.appauth.AuthorizationService;

import java.util.concurrent.ExecutorService;
...

public class TokenActivity extends AppCompatActivity {
    ...
    private AuthorizationService mAuthService;
    private AuthStateManager mStateManager;
    private ExecutorService mExecutor;
    private Configuration mConfig;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);

        mStateManager =
AuthStateManager.getInstance(this);
        mExecutor =
Executors.newSingleThreadExecutor();
        mConfig= Configuration.getInstance(this);

        mAuthService = new AuthorizationService(this,
            new AppAuthConfiguration.Builder()

.setConnectionBuilder(mConfig.mConnectionBuilder)
                .build());

        ...
    }

    @Override
```

```

protected void onStart() {
    super.onStart();
    if (mExecutor.isShutdown()) {
        mExecutor =
Executors.newSingleThreadExecutor();
    }

    ...
}
...
}

```

Process Auth Response

Auth response result from the previous intent would be handled by the `exchangeAuthzCodeBasedOnResultFromAuthBrowser` method where its content would be updated to `AuthStateManager` via the `updateAfterAuthorization` method. If an authorization code is returned, `authorizationResponse.createTokenExchangeRequest()` can be called to prepare the content of the token exchange request to obtain OIDC tokens. Since this is PKCE Authorization Code flow, no client authentication is required when calling token endpoint.

Once token exchange is done, the `handleCodeExchangeResponse` callback method would be triggered, where its response should be updated to `AuthStateManager`. If `AuthStateManager`'s current state of the response result is authorized (`mStateManager.getCurrent().isAuthorized()` is "true"), it implies that there was a recent successful authorization response where at least an access token or an ID token has been returned.

Java

```

...

public class TokenActivity extends AppCompatActivity {
    ...

    @Override
    protected void onStart() {
        ...

        boolean recentUserStateForMobileSSOFound =
managingUserStateForMobileSSO();

        //auth response and auth exception returned from caller intent
        if(!recentUserStateForMobileSSOFound)
            exchangeAuthzCodeBasedOnResultFromAuthBrowser();
    }
    ...
}

```

```

/**
 *
 * obtain auth response from the caller of this intend,
 * if auth response or its exception is not empty, it will save the
response to mStateManager
 * ,or else user would be redirected to login page.
 * If there is no exception and the returned response is not empty,
 * proceed to perform token exchange.
 */
private void exchangeAuthzCodeBasedOnResultFromAuthBrowser() {
    AuthorizationResponse response =
AuthorizationResponse.fromIntent(getIntent());
    AuthorizationException ex =
AuthorizationException.fromIntent(getIntent());

    //update auth response to mStateManager
    if (response != null || ex != null) {
        mStateManager.updateAfterAuthorization(response, ex);
    } else {
        returnToLoginActivity();
    }

    if (response != null && response.authorizationCode != null) {
        // authorization code exchange is required
        mStateManager.updateAfterAuthorization(response, ex);
        exchangeAuthorizationCode(response);
    } else if (ex != null) {
        displayNotAuthorized("Authorization flow failed: " +
ex.getMessage());
    } else {
        returnToLoginActivity();
    }
}

/**
 * Perform token exchange by creating token exchange request
 * based on the given authorization response.
 * @param authorizationResponse response of the auth request
 */
@MainThread
private void exchangeAuthorizationCode(AuthorizationResponse
authorizationResponse) {
    displayLoading("Exchanging authorization code");

    performTokenRequest(authorizationResponse.createTokenExchangeRequest(),
        this::handleCodeExchangeResponse);
}

/**
 * perform token refresh request or token exchange request
 * @param request request object that can be used for token
exchange or token refresh
 * @param callback callback method to process token exchange
response result
 */

```

```

    @MainThread
    private void performTokenRequest(
        TokenRequest request,
        AuthorizationService.TokenResponseCallback callback) {
        mAuthService.performTokenRequest(request,
        NoClientAuthentication.INSTANCE, callback);
    }

    /**
     * Update token response to mStateManager,
     * then call mStateManager.isAuthorized to check if the resulted
     response returned
     *   OIDC tokens with no exception. It is recommended to validate
     the resulted id token.
     * @param tokenResponse response result of token exchange request
     * @param authException
     */
    @WorkerThread
    private void handleCodeExchangeResponse(
        @Nullable TokenResponse tokenResponse,
        @Nullable AuthorizationException authException) {

        mStateManager.updateAfterTokenResponse(tokenResponse,
        authException);
        if (!mStateManager.getCurrent().isAuthorized()) {
            final String message = "Authorization Code exchange failed"
                + ((authException != null) ? authException.error :
                "");

            runOnUiThread(() -> displayNotAuthorized(message));
        } else {
            runOnUiThread(this::displayAuthorized);
        }
    }
    ...
}

```

The `managingUserStateForMobileSSO` method is triggered to check if the mobile user has been successfully authorized recently. The app will skip authorization process for mobile SSO if user state is still active. Click [here](#) to find out more.

Access Resource Server

Use `mStateManager.getCurrent().performActionWithFreshTokens(mAuthService, this::<yourActionToCallResourceServer>)`; when you need to access resource server. This method will ensure fresh OIDC tokens are used (e.g. access token has expired or about to expire but refresh token is still valid) before triggering the resource server via `<yourActionToCallResourceServer>` method. `<yourActionToCallResourceServer>` method must have access token, id token and authorization

exception as the parameters, e.g. `yourActionToCallResourceServer(String accessToken, String idToken, AuthorizationException ex)`.

Below is the snippet of the sample code:

Java

```
/**
 * Demonstrates the use of {@link
AuthState#performActionWithFreshTokens} to call your resource server.
This callback will negotiate a new access
 * token / id token for use in a follow-up action, or provide an
error if this fails.
 */
@MainThread
private void accessResourceServer() {
    displayLoading("Accessing resource server");

    mStateManager.getCurrent().performActionWithFreshTokens(mAuthService,
this::accessResourceServer);
}

private void accessResourceServer(String accessToken, String idToken,
AuthorizationException ex) {
    if (ex != null) {
        Log.e(TAG, "Token refresh failed when accessing resource
server");
        displayNotAuthorized("Failed to get fresh tokens to proceed
");
        return;
    }

    mExecutor.submit(() -> {
        try {

            //below is just a sample code to call resource server
            endpoint using access token as Bearer token
            // ConnectionBuilder connectionBuilder =
            ConnectionBuilder.INSTANCE;
            // HttpURLConnection conn =
            connectionBuilder.openConnection(Uri.parse(yourResourceServerEndpoint.toString()));
            //Log.e("TokenActivity",
            yourResourceServerEndpoint.toString());
            // conn.setRequestProperty("Authorization", "Bearer " +
            accessToken);
            // conn.setInstanceFollowRedirects(false);
            // String response =
            Okio.buffer(Okio.source(conn.getInputStream())).readString(Charset.forName("UTF-8"));
            // mAccessResourceServerResponseJson.set(new
            JSONObject(response));
        }
    });
}
```

```

        } catch (IOException ioEx) {
            Log.e(TAG, "Network error when querying
yourResourceServer endpoint", ioEx);

        } catch (JSONException jsonEx) {
            Log.e(TAG, "Failed to parse yourResourceServer
response");
        }

        runOnUiThread(this::displayAuthorized);
    });

    runOnUiThread(this::displayAuthorized);
}

```

Manage User State for Mobile SSO

OIDC supports mobile SSO by allowing one set of user credentials to access multiple mobile applications where the user is only required to perform authentication once using its credential with the OpenID Provider.

This section will explain the sample codes to use to facilitate mobile SSO. For simplicity of illustration and implementation, Access Token and ID Token are used as the application session. If the recent authentication response was found, mobile user would be allowed to SSO to the mobile app as long as the Access Token and ID Token are still valid (not yet expired). There is a special handling to automatically refresh tokens that are going to be expired prior to server side validation and granting access to user data.

Check User State During Login Activity

If the user has a recent response with a successful authorisation where at least an access token or an ID token exist, (if (mAuthStateManager.getCurrent().isAuthorized())), the app should redirect mobile user to token activity page.

Java

```

...

public class LoginActivity extends AppCompatActivity {
    ...

    @Override
    protected void onCreate(Bundle savedInstanceState)

```

```

{
    ...
    //check if recently authenticated
    if
    (mAuthStateManager.getCurrent().isAuthorized()) {
        Log.i(TAG, "User is already authenticated,
proceeding to token activity");
        startActivity(new Intent(this,
TokenActivity.class));
        finish();
        return;
    }
    ...
}
    ...
}

```

Check User State During Token Activity

If the mobile user has been authorised successfully previously, `mStateManager.getCurrent().performActionWithFreshTokens` would try to get the fresh OIDC token. This is done by retrieving from the cache value of the recent response. But if the access token is expired or about to be expired, the tokens would be obtained from token refresh request - provided user's refresh token is still valid. If user's refresh token has expired, mobile user is expected to be authenticated again in order to access the user's content.

Once the fresh OIDC tokens are obtained, the action to call app server (to validate the ID and access tokens, return of user profile and app context based on user profile) would be triggered, followed by displaying the authorised content corresponding to the user's profile and its application context.

Java

```

...

public class TokenActivity extends AppCompatActivity {
    ...

    @Override
    protected void onStart() {
        ...

        boolean recentUserStateForMobileSSOFound =
managingUserStateForMobileSSO();

        //auth response and auth exception returned from caller

```

```

intent
    if(!recentUserStateForMobileSSOFound)
        exchangeAuthzCodeBasedOnResultFromAuthBrowser();
    }

    /**
     * Check if there is recent successful token response result, if
     yes then try get the fresh tokens from recent auth/token response,
     and call app server to validate tokens and return user profile and
     app context based
     * on user profile.
     *
     */
    private boolean managingUserStateForMobileSSO() {
        //check if SSO login recently
        if (mStateManager.getCurrent().isAuthorized()) {
            //call app server backend to validate tokens and return
            app context by user profile
            validateTokensReturnUserAppContext();
            return true;
        } else {
            return false;
        }
    }

    private void validateTokensReturnUserAppContext() {

mStateManager.getCurrent().performActionWithFreshTokens(mAuthService,
this::validateTokensReturnUserAppContext);
    }

    private void validateTokensReturnUserAppContext(String
accessToken, String idToken, AuthorizationException ex) {
        if (ex != null) {
            Log.e(TAG, "Token refresh failed when
            validateTokensReturnUserAppContext");
            runOnUiThread(() -> displayNotAuthorized("Token refresh
            failed"));
            return;
        }
        //TODO: uncomment the following code to implement the actual
        implementation
        //mExecutor.submit(() -> {
        //    try {
        //        //TODO: call backend server to validate id token and
        introspect access token. It should return user profile and app
        context to the app (for display)
        //        process the response and save user profile and app
        context so that it can be use for display under displayAuthorized
        method
        //    } catch (Exception e) {
        //        Log.e(TAG, "Failed to
        validateTokensReturnUserAppContext", e);
        //    }

        runOnUiThread(this::displayAuthorized);
    }

```

```
        //});  
    }  
}
```

Refer to [TokenActivity](#) for the full source.

Refresh Token (Optional)

AppAuth provides manual and automatic handling for token refresh. Since fresh tokens can be automatically obtained before an action is performed using `mStateManager.getCurrent().performActionWithFreshTokens(mAuthService, this::yourActionToPerform)`, it is not necessary to invoke token refresh manually. Click [here](#) to see the example of how fresh tokens could be obtained before an action is invoked, e.g. accessing resource server.

5. Test/Build OIDC Android APK

Optionally, if you don't want to set up OP endpoints using HTTPS protocol (by yourself) for testing, you can use [ngrok](#) to provide HTTPS tunnelling for AccessMatrix UAM server as the OpenID Provider, so that you can test your OIDC Android APK calling OP's endpoints using HTTPS protocol.

6. Appendices

A. Configuration

Java

```
package com.isprint.appauth.demo;

import android.content.Context;
import android.net.Uri;

import net.openid.appauth.connectivity.ConnectionBuilder;
import net.openid.appauth.connectivity.DefaultConnectionBuilder;

import java.lang.ref.WeakReference;

/**
 * OIDC Client Configuration
 *
 * Note: You are required to set the following mandatory
 settings: discoveryUri, redirectUri, mClientId.
 * Optionally, you may configure mScope to include
 additional scope(s) which should be space delimited.
 */
public final class Configuration {

    //TODO: mandatory settings
    private static final String discoveryUri =
"https://8ed7b0d28b5c.ngrok.io/clp/.well-known/openid-configuration";
    private static final String redirectUri =
"com.isprint.appauth.demo:/com.isprint.appauth.demo";
    static final String mClientId =
"QEVx6ON3n19E0j618aXfuQ";
    static final String mScope = "openid";

    static final ConnectionBuilder mConnectionBuilder =
DefaultConnectionBuilder.INSTANCE;
    //for testing purpose use this if your OP config and
endpoints are using HTTP:
    //static final ConnectionBuilder mConnectionBuilder =
ConnectionBuilderForTesting.INSTANCE;

    static final Uri mRedirectUri;
    static final Uri mDiscoveryUri;

    private static WeakReference<Configuration> sInstance
= new WeakReference<>(null);

    static {
        try {
            mDiscoveryUri = Uri.parse(discoveryUri);
```

```

        } catch (Throwable ex) {
            throw new
IllegalArgumentException("discoveryUri could not be
parsed");
        }

        try {
            mRedirectUri = Uri.parse(redirectUri);
        } catch (Throwable ex) {
            throw new
IllegalArgumentException("discoveryUri could not be
parsed");
        }
    }

    public static com.isprint.appauth.demo.Configuration
getInstance(Context context) {
        com.isprint.appauth.demo.Configuration config =
sInstance.get();
        if (config == null) {
            config = new
com.isprint.appauth.demo.Configuration();
            sInstance = new
WeakReference<Configuration>(config);
        }

        return config;
    }

    private Configuration() {
    }
}

```

B. AuthStateManager

Java

```

package com.isprint.appauth.demo;

import android.content.Context;
import android.content.SharedPreferences;
import android.util.Log;

import net.openid.appauth.AuthState;
import net.openid.appauth.AuthorizationException;
import net.openid.appauth.AuthorizationResponse;
import net.openid.appauth.RegistrationResponse;
import net.openid.appauth.TokenResponse;

import org.json.JSONException;

import java.lang.ref.WeakReference;
import java.util.concurrent.atomic.AtomicReference;
import java.util.concurrent.locks.ReentrantLock;

```

```
import androidx.annotation.AnyThread;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

public class AuthStateManager {

    private static final
AtomicReference<WeakReference<AuthStateManager>>
INSTANCE_REF =
        new AtomicReference<> (new
WeakReference<> (null));

    private static final String TAG =
"AuthStateManager";

    private static final String STORE_NAME =
"AuthState";
    private static final String KEY_STATE = "state";

    private final SharedPreferences mPrefs;
    private final ReentrantLock mPrefsLock;
    private final AtomicReference<AuthState>
mCurrentAuthState;

    @AnyThread
    public static AuthStateManager getInstance(@NonNull
Context context) {
        AuthStateManager manager =
INSTANCE_REF.get().get();
        if (manager == null) {
            manager = new
AuthStateManager(context.getApplicationContext());
            INSTANCE_REF.set(new
WeakReference<> (manager));
        }

        return manager;
    }

    private AuthStateManager(Context context) {
        mPrefs =
context.getSharedPreferences(STORE_NAME,
Context.MODE_PRIVATE);
        mPrefsLock = new ReentrantLock();
        mCurrentAuthState = new AtomicReference<>();
    }

    @AnyThread
    @NonNull
    public AuthState getCurrent() {
        if (mCurrentAuthState.get() != null) {
            return mCurrentAuthState.get();
        }

        AuthState state = readState();
        if (mCurrentAuthState.compareAndSet(null,
```

```
state)) {
    return state;
} else {
    return mCurrentAuthState.get();
}
}

@AnyThread
@NonNull
public AuthState replace(@NonNull AuthState state)
{
    writeState(state);
    mCurrentAuthState.set(state);
    return state;
}

@AnyThread
@NonNull
public AuthState updateAfterAuthorization(
    @Nullable AuthorizationResponse response,
    @Nullable AuthorizationException ex) {
    AuthState current = getCurrent();
    current.update(response, ex);
    return replace(current);
}

@AnyThread
@NonNull
public AuthState updateAfterTokenResponse(
    @Nullable TokenResponse response,
    @Nullable AuthorizationException ex) {
    AuthState current = getCurrent();
    current.update(response, ex);
    return replace(current);
}

@AnyThread
@NonNull
public AuthState updateAfterRegistration(
    RegistrationResponse response,
    AuthorizationException ex) {
    AuthState current = getCurrent();
    if (ex != null) {
        return current;
    }

    current.update(response);
    return replace(current);
}

@AnyThread
@NonNull
private AuthState readState() {
    mPrefsLock.lock();
    try {
        String currentState =
mPrefs.getString(KEY_STATE, null);
```

```

        if (currentState == null) {
            return new AuthState();
        }

        try {
            return
AuthState.jsonDeserialize(currentState);
        } catch (JSONException ex) {
            Log.w(TAG, "Failed to deserialize
stored auth state - discarding");
            return new AuthState();
        }
    } finally {
        mPrefsLock.unlock();
    }
}

@AnyThread
private void writeState(@Nullable AuthState state)
{
    mPrefsLock.lock();
    try {
        SharedPreferences.Editor editor =
mPrefs.edit();
        if (state == null) {
            editor.remove(KEY_STATE);
        } else {
            editor.putString(KEY_STATE,
state.jsonSerializeString());
        }

        if (!editor.commit()) {
            throw new IllegalStateException("Failed
to write state to shared prefs");
        }
    } finally {
        mPrefsLock.unlock();
    }
}
}

```

C. ConnectionBuilderForTesting

Java

```

/*
 * Copyright 2016 The AppAuth for Android Authors. All Rights
Reserved.
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except
 * in compliance with the License. You may obtain a copy of the
License at
 *

```

```
* http://www.apache.org/licenses/LICENSE-2.0
*
* Unless required by applicable law or agreed to in writing,
software distributed under the
* License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES
OR CONDITIONS OF ANY KIND, either
* express or implied. See the License for the specific language
governing permissions and
* limitations under the License.
*/

package com.isprint.appauth.demo;

import android.annotation.SuppressLint;
import android.net.Uri;
import android.util.Log;

import net.openid.appauth.Preconditions;
import net.openid.appauth.connectivity.ConnectionBuilder;

import java.io.IOException;
import java.net.HttpURLConnection;
import java.net.URL;
import java.security.KeyManagementException;
import java.security.NoSuchAlgorithmException;
import java.security.cert.X509Certificate;
import java.util.concurrent.TimeUnit;

import javax.net.ssl.HostnameVerifier;
import javax.net.ssl.HttpsURLConnection;
import javax.net.ssl.SSLContext;
import javax.net.ssl.SSLSession;
import javax.net.ssl.TrustManager;
import javax.net.ssl.X509TrustManager;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

/**
 * An example implementation of {@link ConnectionBuilder} that
permits connecting to http
 * links, and ignores certificates for https connections. *THIS
SHOULD NOT BE USED IN PRODUCTION
 * CODE*. It is intended to facilitate easier testing of AppAuth
against development servers
 * only.
 */
public final class ConnectionBuilderForTesting implements
ConnectionBuilder {

    public static final
com.isprint.appauth.demo.ConnectionBuilderForTesting INSTANCE = new
com.isprint.appauth.demo.ConnectionBuilderForTesting();

    private static final String TAG = "ConnBuilder";

    private static final int CONNECTION_TIMEOUT_MS = (int)
```

```

TimeUnit.SECONDS.toMillis(15);
    private static final int READ_TIMEOUT_MS = (int)
TimeUnit.SECONDS.toMillis(10);

    private static final String HTTP = "http";
    private static final String HTTPS = "https";

    @SuppressWarnings("TrustAllX509TrustManager")
    private static final TrustManager[] ANY_CERT_MANAGER = new
TrustManager[] {
        new X509TrustManager() {
            public X509Certificate[] getAcceptedIssuers() {
                return null;
            }

            public void checkClientTrusted(X509Certificate[]
certs, String authType) {}

            public void checkServerTrusted(X509Certificate[]
certs, String authType) {}
        }
    };

    @SuppressWarnings("BadHostnameVerifier")
    private static final HostnameVerifier ANY_HOSTNAME_VERIFIER =
new HostnameVerifier() {
        public boolean verify(String hostname, SSLSession session)
{
            return true;
        }
    };

    @Nullable
    private static final SSLContext TRUSTING_CONTEXT;

    static {
        SSLContext context;
        try {
            context = SSLContext.getInstance("SSL");
        } catch (NoSuchAlgorithmException e) {
            Log.e("ConnBuilder", "Unable to acquire SSL context");
            context = null;
        }

        SSLContext initializedContext = null;
        if (context != null) {
            try {
                context.init(null, ANY_CERT_MANAGER, new
java.security.SecureRandom());
                initializedContext = context;
            } catch (KeyManagementException e) {
                Log.e(TAG, "Failed to initialize trusting SSL
context");
            }
        }

        TRUSTING_CONTEXT = initializedContext;
    }

```

```

    }

    private ConnectionBuilderForTesting() {
        // no need to construct new instances
    }

    @NonNull
    @Override
    public HttpURLConnection openConnection(@NonNull Uri uri)
    throws IOException {
        Preconditions.checkNotNull(uri, "url must not be null");
        Preconditions.checkArgument(HTTP.equals(uri.getScheme()) ||
        HTTPS.equals(uri.getScheme()),
            "scheme or uri must be http or https");
        HttpURLConnection conn = (HttpURLConnection) new
        URL(uri.toString()).openConnection();
        conn.setConnectTimeout(CONNECTION_TIMEOUT_MS);
        conn.setReadTimeout(READ_TIMEOUT_MS);
        conn.setInstanceFollowRedirects(false);

        if (conn instanceof HttpsURLConnection && TRUSTING_CONTEXT
        != null) {
            HttpsURLConnection httpsConn = (HttpsURLConnection)
            conn;

            httpsConn.setSSLSocketFactory(TRUSTING_CONTEXT.getSocketFactory());
            httpsConn.setHostnameVerifier(ANY_HOSTNAME_VERIFIER);
        }

        return conn;
    }
}

```

D. LoginActivity

Java

```

package com.isprint.appauth.demo;

import android.app.PendingIntent;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;

import net.openid.appauth.AppAuthConfiguration;
import net.openid.appauth.AuthState;
import net.openid.appauth.AuthorizationException;
import net.openid.appauth.AuthorizationRequest;
import net.openid.appauth.AuthorizationService;
import net.openid.appauth.AuthorizationServiceConfiguration;
import net.openid.appauth.ResponseTypeValues;
import net.openid.appauth.browser.BrowserAllowList;

```

```
import net.openid.appauth.browser.VersionedBrowserMatcher;

import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicReference;

import androidx.annotation.MainThread;
import androidx.appcompat.app.AppCompatActivity;
import androidx.browser.customtabs.CustomTabsIntent;

public class LoginActivity extends AppCompatActivity {
    private static final String TAG = "LoginActivity";
    private Configuration mConfig;
    private AuthorizationService mAuthService;
    private AuthStateManager mAuthStateManager;
    private final AtomicReference<AuthorizationRequest> mAuthRequest =
new AtomicReference<>();
    private final AtomicReference<CustomTabsIntent> mAuthIntent = new
AtomicReference<>();
    private ExecutorService mExecutor;
    private CountDownLatch mAuthIntentLatch = new CountDownLatch(1);
    private static final String EXTRA_FAILED = "failed";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        mExecutor = Executors.newSingleThreadExecutor();
        mConfig = Configuration.getInstance(this);
        mAuthStateManager = AuthStateManager.getInstance(this);

        //check if recently authenticated
        if (mAuthStateManager.getCurrent().isAuthorized()) {
            Log.i(TAG, "User is already authenticated, proceeding to
token activity");
            startActivity(new Intent(this, TokenActivity.class));
            finish();
            return;
        }

        setContentView(R.layout.activity_login);

        Button loginBtn = (Button) findViewById(R.id.loginBtn);
        loginBtn.setClickable(true);
        loginBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                startAuth();
            }
        });

        Log.i(TAG, "Initializing environment for authentication");

        mExecutor.submit(this::initializeEnvironmentPrepareAuthRequest);
    }
}
```

```
@Override
protected void onStart() {
    super.onStart();
    if (mExecutor.isShutdown()) {
        mExecutor = Executors.newSingleThreadExecutor();
    }
}

@Override
protected void onStop() {
    super.onStop();
    mExecutor.shutdownNow();
}

@Override
protected void onDestroy() {
    super.onDestroy();

    if (mAuthService != null) {
        mAuthService.dispose();
    }
}

private void initializeEnvironmentPrepareAuthRequest() {

    recreateAuthorizationService();

    if
(mAuthStateManager.getCurrent().getAuthorizationServiceConfiguration()
!= null) {
        Log.i(TAG, "App was running not long ago, auth config
already established");
        initializeClientAndAuthRequest();
        return;
    }

    Log.i(TAG, "Retrieving OpenID discovery doc");
    AuthorizationServiceConfiguration.fetchFromUrl(
        mConfig.mDiscoveryUri,
        this::handleConfigurationRetrievalResult,
        mConfig.mConnectionBuilder);
}

private void recreateAuthorizationService() {
    if (mAuthService != null) {
        Log.i(TAG, "Discarding existing AuthService instance");
        mAuthService.dispose();
    }
    mAuthService = createAuthorizationService();
    mAuthRequest.set(null);
    mAuthIntent.set(null);
}

private AuthorizationService createAuthorizationService() {
    Log.i(TAG, "Creating authorization service");
```

```

        AppAuthConfiguration.Builder builder = new
AppAuthConfiguration.Builder();
        builder.setBrowserMatcher(new BrowserAllowList(
            VersionedBrowserMatcher.CHROME_CUSTOM_TAB,
            VersionedBrowserMatcher.SAMSUNG_CUSTOM_TAB));
        builder.setConnectionBuilder(mConfig.mConnectionBuilder);

        //In Production: need to set this to false
        builder.setSkipIssuerHttpsCheck(true);
        builder.setSkipNonceVerification(false);

        return new AuthorizationService(this, builder.build());
    }

    private void initializeClientAndAuthRequest() {
        Log.i(TAG, "client ID: " + mConfig.mClientId);
        runOnUiThread(this::initializeAuthRequest);
        return;
    }

    private void initializeAuthRequest() {
        createAuthRequest();
        warmUpBrowser();
    }

    private void createAuthRequest() {
        Log.i(TAG, "Preparing auth request");
        AuthorizationRequest.Builder authRequestBuilder = new
AuthorizationRequest.Builder(
mAuthStateManager.getCurrent().getAuthorizationServiceConfiguration(),
        mConfig.mClientId,
        ResponseTypeValues.CODE,
        mConfig.mRedirectUri)
            .setScope(mConfig.mScope);

        mAuthRequest.set(authRequestBuilder.build());
    }

    private void warmUpBrowser() {
        mAuthIntentLatch = new CountDownLatch(1);
        mExecutor.execute(() -> {
            Log.i(TAG, "Warming up browser instance for auth request");
            CustomTabsIntent.Builder intentBuilder =

mAuthService.createCustomTabsIntentBuilder(mAuthRequest.get().toUri());
            mAuthIntent.set(intentBuilder.build());
            mAuthIntentLatch.countDown();
        });
    }

    private void startAuth () {
        mExecutor.submit(this::doAuth);
    }

    private void doAuth() {

        try {

```

```

        //this is to make sure custom tab is ready before proceed
        with auth request
        mAuthIntentLatch.await();
    } catch (InterruptedException ex) {
        Log.w(TAG, "Interrupted while waiting for auth intent");
    }

    //the intent to be sent when the flow completes.
    Intent completionIntent = new Intent(this,
TokenActivity.class);
    //the intent to be sent when the flow cancels.
    Intent cancelIntent = new Intent(this, LoginActivity.class);
    cancelIntent.putExtra(EXTRA_FAILED, true);
    cancelIntent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);

    mAuthService.performAuthorizationRequest(
        mAuthRequest.get(),
        PendingIntent.getActivity(this, 0, completionIntent,
0),
        PendingIntent.getActivity(this, 0, cancelIntent, 0),
        mAuthIntent.get());
    }

    @MainThread
    private void handleConfigurationRetrievalResult(
        AuthorizationServiceConfiguration config,
        AuthorizationException ex) {
        if (config == null) {
            Log.i(TAG, "Failed to retrieve discovery document", ex);
            return;
        }

        Log.i(TAG, "Discovery document retrieved");
        mAuthStateManager.replace(new AuthState(config));
        mExecutor.submit(this::initializeClientAndAuthRequest);
    }
}

```

E. TokenActivity

Java

```

package com.isprint.appauth.demo;

import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

import net.openid.appauth.AppAuthConfiguration;
import net.openid.appauth.AuthState;
import net.openid.appauth.AuthorizationException;

```

```
import net.openid.appauth.AuthorizationResponse;
import net.openid.appauth.AuthorizationService;
import net.openid.appauth.NoClientAuthentication;
import net.openid.appauth.TokenRequest;
import net.openid.appauth.TokenResponse;

import org.joda.time.DateTimeZone;
import org.joda.time.format.DateTimeFormat;
import org.joda.time.format.DateTimeFormatter;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

import androidx.annotation.MainThread;
import androidx.annotation.Nullable;
import androidx.annotation.WorkerThread;
import androidx.appcompat.app.AppCompatActivity;

public class TokenActivity extends AppCompatActivity {
    private static final String TAG = "TokenActivity";

    private AuthorizationService mAuthService;
    private AuthStateManager mStateManager;
    private ExecutorService mExecutor;
    private Configuration mConfig;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        mStateManager = AuthStateManager.getInstance(this);
        mExecutor = Executors.newSingleThreadExecutor();
        mConfig = Configuration.getInstance(this);

        mAuthService = new AuthorizationService(this,
            new AppAuthConfiguration.Builder()
                .setConnectionBuilder(mConfig.mConnectionBuilder)
                .build());

        setContentView(R.layout.activity_token);
    }

    @Override
    protected void onStart() {
        super.onStart();
        if (mExecutor.isShutdown()) {
            mExecutor = Executors.newSingleThreadExecutor();
        }

        boolean recentUserStateForMobileSSOFound =
            managingUserStateForMobileSSO();

        //auth response and auth exception returned from caller intent
        if(!recentUserStateForMobileSSOFound)
            exchangeAuthzCodeBasedOnResultFromAuthBrowser();
    }
}
```

```

@Override
protected void onSaveInstanceState(Bundle state) {
    super.onSaveInstanceState(state);
}

@Override
protected void onDestroy() {
    super.onDestroy();
    mAuthService.dispose();
    mExecutor.shutdownNow();
}

/**
 * Check if there is recent successful token response result, if yes
 * then try get the fresh tokens from recent auth/token response, and call
 * app server to validate tokens and return user profile and app context
 * based
 * on user profile.
 */
private boolean managingUserStateForMobileSSO() {
    //check if SSO login recently
    if (mStateManager.getCurrent().isAuthorized()) {
        //call app server backend to validate tokens and return app
        context by user profile
        validateTokensReturnUserAppContext();
        return true;
    } else {
        return false;
    }
}

private void validateTokensReturnUserAppContext() {
    mStateManager.getCurrent().performActionWithFreshTokens(mAuthService,
    this::validateTokensReturnUserAppContext);
}

private void validateTokensReturnUserAppContext(String accessToken,
String idToken, AuthorizationException ex) {
    if (ex != null) {
        Log.e(TAG, "Token refresh failed when
        validateTokensReturnUserAppContext");
        runOnUiThread(() -> displayNotAuthorized("Token refresh
        failed"));
        return;
    }
    //TODO: uncomment the following code to implement the actual
    implementation
    //mExecutor.submit(() -> {
    //    try {
    //        //TODO: call backend server to validate id token and
    //        introspect access token. It should return user profile and app context to
    //        the app (for display)
    //        // process the response and save user profile and app

```

```

context so that it can be use for display under displayAuthorized method
        // } catch (Exception e) {
        //     Log.e(TAG, "Failed to
validateTokensReturnUserAppContext", e);
        //}

        runOnUiThread(this::displayAuthorized);
    //});
}

/**
 *
 * obtain auth response from the caller of this intend,
 * if auth response or its exception is not empty, it will save the
response to mStateManager
 * ,or else user would be redirected to login page.
 * If there is no exception and the returned response is not empty,
 * proceed to perform token exchange.
 */
@MainThread
private void exchangeAuthzCodeBasedOnResultFromAuthBrowser() {
    AuthorizationResponse response =
AuthorizationResponse.fromIntent(getIntent());
    AuthorizationException ex =
AuthorizationException.fromIntent(getIntent());

    //update auth response to mStateManager
    if (response != null || ex != null) {
        mStateManager.updateAfterAuthorization(response, ex);
    } else {
        returnToLoginActivity();
    }

    if (response != null && response.authorizationCode != null) {
        // authorization code exchange is required
        mStateManager.updateAfterAuthorization(response, ex);
        exchangeAuthorizationCode(response);
    } else if (ex != null) {
        displayNotAuthorized("Authorization flow failed: " +
ex.getMessage());
    } else {
        returnToLoginActivity();
    }
}

/**
 * Perform token exchange by creating token exchange request
 * based on the given authorization response.
 * @param authorizationResponse response of the auth request
 */
@MainThread
private void exchangeAuthorizationCode(AuthorizationResponse
authorizationResponse) {
    displayLoading("Exchanging authorization code");

    performTokenRequest(authorizationResponse.createTokenExchangeRequest(),
        this::handleCodeExchangeResponse);
}

```

```

    }

    @MainThread
    private void displayLoading(String message) {

        findViewById(R.id.loadingContainerLayout).setVisibility(View.VISIBLE);
        findViewById(R.id.authorizedLayout).setVisibility(View.GONE);
        findViewById(R.id.unauthorizedLayout).setVisibility(View.GONE);

        ((TextView) findViewById(R.id.loadContainerTxtView)).setText(message);
    }

    /**
     * perform token refresh request or token exchange request
     * @param request request object that can be used for token exchange
     or token refresh
     * @param callback callback method to process token exchange response
     result
     */
    @MainThread
    private void performTokenRequest(
        TokenRequest request,
        AuthorizationService.TokenResponseCallback callback) {
        mAuthService.performTokenRequest(request,
        NoClientAuthentication.INSTANCE, callback);
    }

    /**
     * Update token response to mStateManager,
     * then call mStateManager.isAuthorized to check if the resulted
     response returned
     * OIDC tokens with no exception. It is recommended to validate the
     resulted id token.
     * @param tokenResponse response result of token exchange request
     * @param authException
     */
    @WorkerThread
    private void handleCodeExchangeResponse(
        @Nullable TokenResponse tokenResponse,
        @Nullable AuthorizationException authException) {

        mStateManager.updateAfterTokenResponse(tokenResponse,
        authException);
        if (!mStateManager.getCurrent().isAuthorized()) {
            final String message = "Authorization Code exchange failed"
                + ((authException != null) ? authException.error :
                "");

            runOnUiThread(() -> displayNotAuthorized(message));
        } else {
            runOnUiThread(this::displayAuthorized);
        }
    }

    @MainThread
    private void displayAuthorized() {

```

```

        findViewById(R.id.authorizedLayout).setVisibility(View.VISIBLE);
        findViewById(R.id.unauthorizedLayout).setVisibility(View.GONE);

findViewById(R.id.loadingContainerLayout).setVisibility(View.GONE);
AuthState state = mStateManager.getCurrent();

    TextView refreshTokenInfoView =
findViewById(R.id.refreshTokenInfoTxtView);
    refreshTokenInfoView.setText((state.getRefreshToken() == null)
        ? R.string.refreshToken_not_returned
        : R.string.refreshToken_returned);

    TextView idTokenInfoView = (TextView)
findViewById(R.id.idTokenInfoTxtView);
    idTokenInfoView.setText((state.getIdToken()) == null
        ? R.string.idToken_not_returned
        : R.string.idToken_returned);

    TextView accessTokenInfoView = (TextView)
findViewById(R.id.accessTokenInfoTxtView);
    if (state.getAccessToken() == null) {

accessTokenInfoView.setText(R.string.accessToken_not_returned);
        } else {
            Long expiresAt = state.getAccessTokenExpirationTime();
            if (expiresAt == null) {

accessTokenInfoView.setText(R.string.accessTokenExpiry_not_returned);
                } else if (expiresAt < System.currentTimeMillis()) {

accessTokenInfoView.setText(R.string.accessToken_expired);
                    } else {
                        String template =
getResources().getString(R.string.accessToken_expires_at);
                        DateFormatter fmt = DateTimeFormat.forPattern("yyyy-
MM-dd HH:mm:ss ZZ").withZone(DateTimeZone.forID("Asia/Singapore"));
                        accessTokenInfoView.setText(String.format(template,
                            fmt.print(expiresAt)));
                    }
                }
        }
    }

    @MainThread
    private void displayNotAuthorized(String explanation) {

findViewById(R.id.unauthorizedLayout).setVisibility(View.VISIBLE);
        findViewById(R.id.authorizedLayout).setVisibility(View.GONE);

findViewById(R.id.loadingContainerLayout).setVisibility(View.GONE);

        ((TextView) findViewById(R.id.unauthorizedExplanationTxtView)).setText(explanation);
        findViewById(R.id.retryBtn).setOnClickListener((View view) ->

```

```
returnToLoginActivity());
    }

    @MainThread
    private void returnToLoginActivity() {

        AuthState currentState = mStateManager.getCurrent();
        AuthState clearedState =
            new
AuthState(currentState.getAuthorizationServiceConfiguration());

        mStateManager.replace(clearedState);

        Intent mainIntent = new Intent(this, LoginActivity.class);
        mainIntent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
        startActivity(mainIntent);
        finish();
    }

}
```
